

Comprehensive Engineering Report

Industrial Conveyor Physics Simulation, WebRTC Streaming & React Control Architecture

Executive Summary: This technical dossier provides an exhaustive overview of the end-to-end development, architecture, physics instrumentation, and real-time streaming bridge engineered in this workspace. The system connects an **Omniverse Kit headless GPU rendering server** with a modern **React (Vite + TypeScript) client** over bidirectional WebRTC data and media channels, delivering an interactive industrial conveyor digital twin with live box counting, surface velocity physics, and dynamic telemetry.

1. Project Scope & Architecture Evolution

The initiative commenced as a bidirectional proof-of-concept allowing a web client to render a remote Kit viewport and spawn simple 3D primitives over a WebRTC data channel. Through a series of structured engineering sprints, the system was expanded into a high-fidelity **industrial digital twin** featuring realistic PhysX mechanics, dynamic asset authoring, and asynchronous push telemetry.

Phase 1: Bidirectional WebRTC Core

Established low-latency H.264 video streaming via `@nvidia/ov-web-rtc` AppStreamer, Kit event dispatching, and custom JSON data channels without external REST relays.

Phase 2: Conveyor Physics & Digital Twin

Broke out procedural and baked 3D conveyor assets, PhysX Surface Velocity belt mechanics, collision geometry, gravity timelines, and collection bin containment.

Phase 3: Real-Time Sensor Telemetry

Engineered continuous spatial arrival detection for delivered packages, spontaneous push notifications to the browser, and live box counter synchronization.

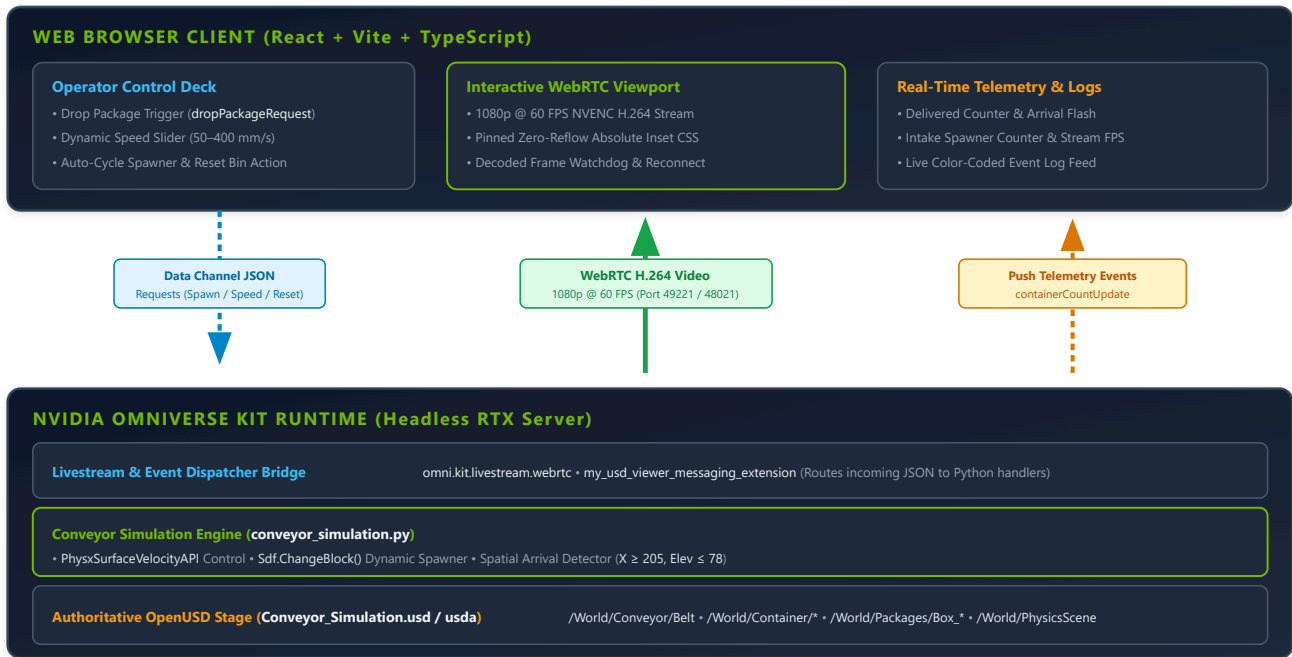
Phase 4: Resilience & Production Hardening

Implemented hardware H.264 codec pinning, Kit run-loop pacing (60 FPS), dynamic aspect ratio resizing, a decoded-frame watchdog, and instant stream reconnection.

2. Visual System Architecture & Streaming Topology

The diagram below visualizes the client-server boundaries, media channels, signaling routes, and data-flow interactions between the browser frontend and Omniverse Kit.

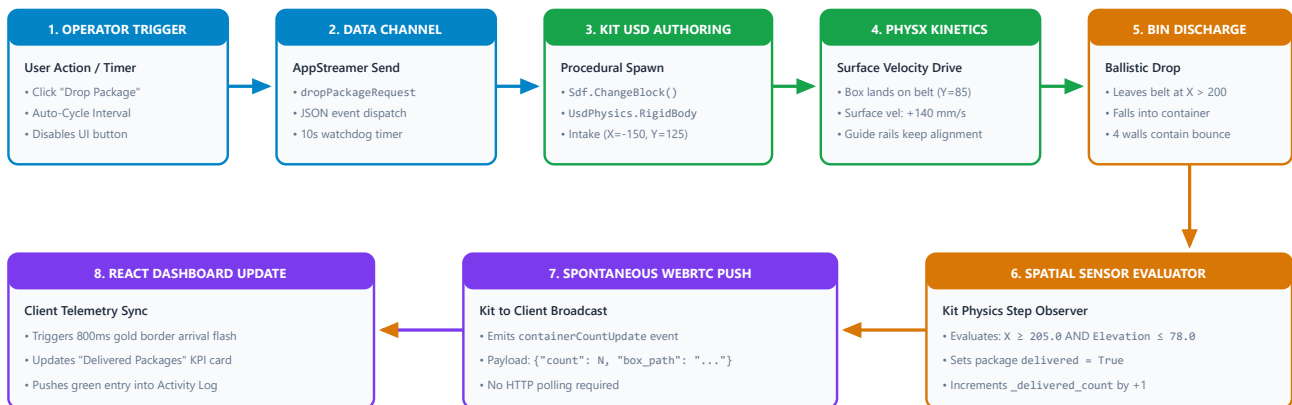
FIGURE 1: BIDIRECTIONAL WEBRTC SYSTEM TOPOLOGY & COMPONENT HIERARCHY



3. Interactive Package Lifecycle & Sensor Detection Flowchart

The step-by-step logic loop governing package ingestion, kinetic belt transfer, ballistic discharge, spatial detection, and telemetry dispatch is detailed in the flowchart below.

FIGURE 2: RIGID BODY PACKAGE DISPATCH & TELEMETRY FEEDBACK LOOP



4. USD Hierarchy & Physics Modeling

The simulation environment is authored natively in OpenUSD (`Conveyor_Simulation.usda` and binary `.usd`). The stage operates in standard centimeter units (`metersPerUnit = 0.01`) with a Y-Up coordinate convention, fully configured with realistic dynamic friction and mass distribution.

Prim Path	USD Type & Schema	Dimensions / Transforms	Functional Role in Simulation
<code>/World/PhysicsScene</code>	<code>UsdPhysics.Scene</code>	Gravity: <code>(0, -980, 0)</code> cm/s ²	Defines global dynamic simulation parameters and gravitational vector.
<code>/World/Conveyor/Belt</code>	<code>Cube</code> + <code>PhysxSurfaceVelocityAPI</code>	Scale: <code>(400, 10, 80)</code> , Elev: 80 cm	Active moving belt. Imparts linear velocity along the +X axis (default 140 mm/s) to resting packages.
<code>/World/Conveyor/Leg_0..2</code>	<code>Cube</code> + <code>PhysicsCollisionAPI</code>	Scale: <code>(12, 75, 76)</code> at X=-160, 0, +160	Heavy structural steel frame legs holding the conveyor bed at working height.
<code>/World/Conveyor/Rail_0..1</code>	<code>Cube</code> + <code>PhysicsCollisionAPI</code>	Scale: <code>(400, 22, 6)</code> at Z=±44 cm	Dual lateral side guide rails ensuring packages do not veer off under acceleration.
<code>/World/Container/Floor</code>	<code>Cube</code> + <code>PhysicsCollisionAPI</code>	Scale: <code>(120, 6, 120)</code> at X=275 cm	Discharge receptacle base positioned below the head pulley to collect dropped packages.
<code>/World/Container/Walls</code>	<code>Cube</code> (4 Perimeter Walls)	Height: 55 cm, Safety Amber	Enclosed bin perimeter preventing accumulated packages from scattering onto the factory floor.
<code>/World/Packages/Box_*</code>	<code>Cube</code> + <code>RigidBodyAPI</code> + <code>MassAPI</code>	Size: 32 cm ³ , Mass: 2.0 kg	Dynamic package with random vibrant palette (gold, blue, neon green, coral orange, teal).

5. WebRTC Data Channel Messaging Contract

Communication between the React web client and Omniverse Kit occurs over named bidirectional JSON messages routed through the streaming data channel.

Event Identifier	Direction	Payload Structure	Description
<code>dropPackageRequest</code>	Browser → Kit	<code>{}</code>	Requests the server to spawn and drop a rigid-body package onto the conveyor intake.
<code>dropPackageResult</code>	Kit → Browser	<code>{"result": "success", "path": "/World/Packages/Box_1"}</code>	Confirms package creation and passes the resulting USD prim path.
<code>setConveyorSpeedRequest</code>	Browser → Kit	<code>{"speed": 220.0}</code>	Dynamically updates <code>PhysxSurfaceVelocityAPI</code> on the moving belt in real time.
<code>setConveyorSpeedResult</code>	Kit → Browser	<code>{"result": "success", "speed": 220.0}</code>	Confirms that the new surface speed has been applied in PhysX.
<code>containerCountUpdate</code>	Kit → Browser	<code>{"count": 7, "box_path": "...", "reset": false}</code>	Spontaneous push notification emitted immediately when a package enters the bin.
<code>resetContainerRequest</code>	Browser → Kit	<code>{}</code>	Purges all active package prims from <code>/World/Packages</code> and zeroes the counter.

6. Production-Grade Performance Tuning & Anti-Freeze Protections

Real-time WebRTC streaming of high-density 3D viewports is susceptible to network jitter, decoder stalls, and frame-pacing irregularities. Six targeted optimizations were incorporated:

1. Hardware H.264 Codec Pinning

Explicitly forces `VideoCodec.H264` on both client and Kit to prevent browser fallback to software VP8/VP9 decoders which cause severe frame stutter.

2. Kit Run-Loop Pacing

Enabled `rateLimitEnabled = true` with `rateLimitFrequency = 60` in Kit to eliminate render jitter and prevent NVENC encoder pipeline starvation.

3. Dynamic Viewport Resizing

Configured `dynamicResize = true` in `my_usd_viewer_streaming.kit` so remote rendering resolutions dynamically adapt to browser dimensions.

4. Sdf.ChangeBlock() Coalescing

All procedural USD scene mutations are batched in change blocks to ensure Hydra constructs acceleration structures once rather than per-attribute.

5. Decoded Frame Watchdog

The React client interrogates `getVideoPlaybackQuality().totalVideoFrames`. If decoded frames halt while connected, a warning and auto-reconnect are triggered.

6. Zero-Reflow Pinned Layout

The HTML5 video container is styled with absolute inset positioning to isolate 60 FPS video frame repaints from the React DOM tree, eliminating layout thrashing.

7. Execution & Operational Verification

The complete platform can be launched locally on Windows 10/11 with an NVIDIA RTX GPU using two isolated terminal sessions:

Step A: Start the Omniverse Kit Headless Streaming Server

```
cd d:\Omniverse_learning\kit-app-template
& ".\_build\windows-x86_64\release\kit\kit.exe" `
  ".\_build\windows-x86_64\release\apps\my_company.my_usd_viewer_streaming.kit" `
  --no-window `
  "--/app/auto_load_usd=..\Conveyor_Simulation.usd"
```

Server initialization completes when `[Info] RTX ready` and `app ready` appear in stdout.

Step B: Start the React Web Dashboard

```
cd d:\Omniverse_learning\Poc_test
npm install
npm run dev -- --host 127.0.0.1
```

Navigate to `http://127.0.0.1:5173/` in Google Chrome or Microsoft Edge.